

SIMATS ENGINEERING

SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

CHENNAI – 602105

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE

Course Code:	DSA05	Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2	Assessment:	Tool 2 – Schema Analysis and Viva Voce
Weightage:		Total Marks:	20 Marks
CO2 Statement: Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)			
Student Name:	U.Ananya	Reg. No.:	192424382
Section / Batch:	AI&DS / 24 batch	Date of Submission:	03-08-2026
Faculty In charge:	K. Veena Devi	Project Mode:	Individual Submission

Code of Conduct

I U.Ananya(192424382) (Name / Reg No) certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: __U.Ananya__

Identification of Entities and Attributes (4 Marks)	Understanding of Relationships (4 Marks)	Analysis of Keys and Constraints (4 Marks)	Detection of Design Issues (4 Marks)	Viva Voce (4 Marks)	Total (20 Marks)
--	---	---	---	----------------------------	-------------------------

--	--	--	--	--	--

1. University Course Registration System

Q1. Why is the Enrollment table necessary instead of storing CourseID directly in the Student table?

Answer:

- A student can enroll in multiple courses, and each course can have multiple students.
- This creates a many-to-many relationship between the Student and Course tables.
- The Enrollment table acts as a bridge (junction) table to manage this relationship.
- It avoids data redundancy and maintains database normalization.
- It also stores additional information such as the Semester for each enrollment.

Q2. If the university wants to track grades for each enrolled course, how would you modify the schema?

Answer:

Add a new column Grade to the Enrollment table.

Modified Enrollment Table:

StudentID	CourseID	Semester	Grade
101	C101	Semester 1	A
102	C102	Semester 1	B+
103	C101	Semester 1	A+

Modified Schema:

Enrollment(StudentID, CourseID, Semester, Grade)

This allows the university to store the grade obtained by each student in every enrolled course while maintaining the relationship between students and courses.

2. Hospital Management System

Q1. What problems might occur if prescription details are stored directly in the

Appointment table?

Answer:

- One appointment may require multiple medicines, but storing them in the Appointment table makes it difficult to manage.
- It causes data redundancy if appointment details are repeated for each medicine.
- Updating or deleting medicine information becomes difficult and may lead to data inconsistency.
- It violates database normalization principles.
- Keeping prescriptions in a separate Prescription table improves data organization and maintenance.

Q2. How would the schema change if a prescription contains multiple medicines?

Answer:

Create a separate Prescription_Medicine table to store multiple medicines for each prescription.

Modified Schema:

Patient(PatientID, Name, Age)

Doctor(DoctorID, DoctorName, Specialization)

Appointment(AppointmentID, PatientID, DoctorID, Date)

Prescription(PrescriptionID, AppointmentID)

Prescription_Medicine(PrescriptionID, Medicine)

Example:

PrescriptionID	Medicine
P101	Paracetamol
P101	Vitamin C
P101	Antibiotic

This design allows one prescription to contain multiple medicines without duplicating appointment information and follows database normalization principles.

3. Online Shopping System

Q1. Why is the OrderDetails table required in this schema?

Answer:

- An order can contain multiple products, and a product can appear in multiple orders.
- This creates a many-to-many relationship between the Order and Product tables.
- The OrderDetails table acts as a bridge (junction) table to manage this relationship.
- It stores the Quantity of each product in an order.
- It reduces data redundancy and maintains database normalization.

Q2. What would happen if ProductID were stored directly in the Order table?

Answer:

- Each order could store only one product, which is not suitable for online shopping.
- To store multiple products, the order information would have to be repeated, causing data redundancy.
- It would lead to data inconsistency and make updates difficult.
- The relationship between orders and products would not be properly represented.
- Therefore, the OrderDetails table is necessary to store multiple products for a single order efficiently.

4. Library Management System

Q1. Explain the relationship between Member and Book through the Issue table.

Answer:

- A member can borrow multiple books.
- A book can be borrowed by different members at different times.
- This creates a many-to-many relationship.
- The Issue table acts as a bridge (junction) table between Member and Book.
- It stores borrowing details such as IssueDate and ReturnDate.

Q2. How would you modify the schema to track overdue fines?

Answer:

Add a Fine column to the Issue table.

Modified Schema:

Issue(MemberID, BookID, IssueDate, ReturnDate, Fine)

5. Banking System

Q1. Why should transactions be stored separately from account details?

Answer:

- One account can have many transactions.
- Keeping transactions separate avoids data redundancy.
- It maintains the complete transaction history.
- Updates become easier and more efficient.
- It follows database normalization principles.

Q2. How would you redesign the schema to support joint accounts?

Answer:

Create a bridge table:

Customer_Account(CustomerID, AccountNo)

This allows multiple customers to share the same account.

6. Employee Payroll System

Q1. Why is Department information separated into a different table?

Answer:

- One department can have many employees.
- Avoids repeating department information.
- Reduces data redundancy.
- Makes updates easier.
- Maintains database normalization.

Q2. What redundancy issues would arise if department details were stored in Employee records?

Answer:

- Department name would be repeated for every employee.
- Wastes storage space.
- Updating department information becomes difficult.

- May cause inconsistent data.
- Violates normalization principles.

7. Hotel Reservation System

Q1. Why should reservation details be maintained separately from room details?

Answer:

- A room can have many reservations at different times.
- Reservation details change frequently.
- Avoids duplicate room information.
- Makes booking management easier.
- Supports multiple customers over time.

Q2. How would you prevent double booking of rooms using the schema?

Answer:

- Check if the room is already booked for the selected dates.
- Allow booking only if no overlapping reservation exists.
- Use RoomID with CheckIn and CheckOut dates to validate availability.

8. Food Delivery Application

Q1. Why is Delivery maintained as a separate table?

Answer:

- Separates order details from delivery information.
- One delivery agent can deliver many orders.
- Makes assigning delivery agents easier.
- Reduces redundancy.
- Improves database organization.

Q2. How would the schema change if multiple delivery agents could handle a single order?

Answer:

Create a bridge table:

Delivery(OrderID, AgentID)

This supports many-to-many relationships between orders and delivery agents.

9. Learning Management System (LMS)

Q1. Why is Enrollment considered a bridge table?

Answer:

- Students can enroll in many courses.
- A course can have many students.
- It connects Student and Course tables.
- Represents a many-to-many relationship.
- Stores enrollment details for each student.

Q2. How would you modify the schema to record assignment grades?

Answer:

Create a new table:

Assignment_Grade(StudentID, AssignmentID, Grade)

or add a Grade column if each assignment belongs to one student.

10. Bus Ticket Reservation System

Q1. Why should booking details be stored separately from passenger information?

Answer:

- One passenger can make multiple bookings.
- Booking details change for each journey.
- Avoids duplicate passenger information.
- Makes booking management easier.
- Supports database normalization.

Q2. If a booking contains multiple passengers, how would you redesign the schema?

Answer:

Create a bridge table:

Booking_Passenger(BookingID, PassengerID)

This allows one booking to include multiple passengers while also allowing a passenger to appear in different bookings if needed.